

Project Report: Expense Tracker & Budget Planning Application

Course: ITM Flutter Mini-Project (Mobile Application Development)

Group 01:- Viketh Hegde, Sahil Kadam, Kishan Ojha, Ashmit Suryavanshi

GitHub Repository: [Finance Tracker](#)

Project Folder: `itm-flutter-finance-tracker`

Video Drive Link: [Flutter video](#)

Date: September 15, 2026

1. Executive Summary

The **Expense Tracker & Budget Planning Application** is a modern, offline-first personal financial management application built using the Flutter framework and Dart. Designed with a sleek **Slice UPI neo-banking pitch-dark aesthetic**, the app provides users with intuitive tools to manage incomes, track categorized expenses, set monthly spending budgets, and visualize their financial health through interactive analytical charts.

The application operates locally without external backend dependencies, ensuring zero-latency responsiveness and total user data privacy through an embedded SQLite database.

2. Objectives & Scope

2.1 Key Objectives

- **Real-Time Financial Tracking:** Enable users to effortlessly log income and expenditure entries with merchant names, categories, notes, and custom timestamps.
- **Budgeting & Spending Discipline:** Provide monthly category-specific budget allocation with visual indicators that notify users when spending approaches or exceeds targets.
- **Interactive Analytics:** Offer graphical breakdowns of spending distributions and half-year trends via chart components.
- **Transaction Safety & Reversion:** Support full CRUD functionality with safe transaction deletion that dynamically recalculates and restores net balances and category totals.

- **Modern User Experience:** Deliver an ultra-premium neo-banking dark UI inspired by Slice UPI, complete with smooth animations, high-contrast typography, and floating dock navigation.

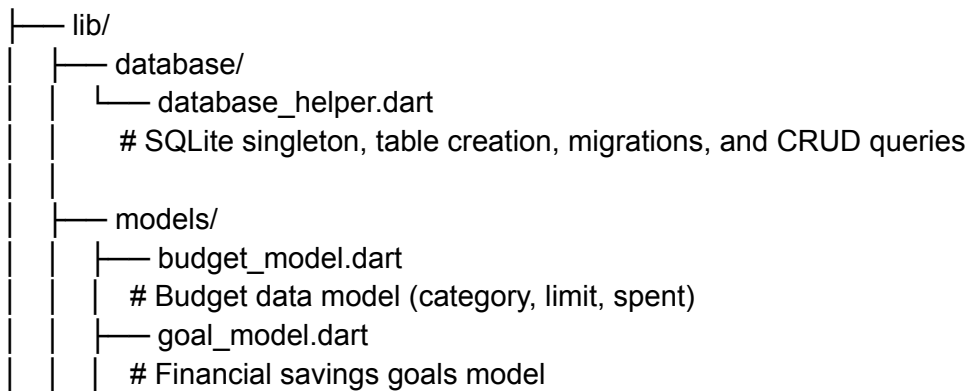
3. Technology Stack

Layer	Technology	Description
Framework	Flutter (v3.x+)	Multiplatform UI toolkit for iOS, Android, macOS, and Web
Language	Dart (v3.x+)	Strongly typed, reactive object-oriented language
Local Database	<code>sqflite</code> / <code>sqflite_common_ffi</code>	Persistent, ACID-compliant local SQL database engine
Data Visualization	<code>fl_chart</code>	Interactive, animated charts for spending and trends
Internationalization	<code>intl</code>	Date formatting, currency parsing, and number localization
Architecture	Layered MVC / Repository Pattern	Clear separation between data models, database helpers, screens, and reusable UI components

4. System Architecture & Directory Structure

The project code is organized inside `itm-flutter-finance-tracker` using a modular and maintainable folder structure:

itm-flutter-finance-tracker/



```
├── transaction_model.dart
│   # Transaction model (amount, type, category, date, merchant)
├── screens/
│   ├── dashboard_screen.dart
│   │   # Hero balance card, quick actions, operations list
│   ├── transactions_screen.dart
│   │   # Filterable transaction history with search & swipe actions
│   ├── add_transaction_screen.dart
│   │   # Form for adding Income and Expenses with validation
│   ├── transaction_detail_screen.dart
│   │   # Modal sheet showing receipt metadata & 6-month chart
│   ├── budgets_screen.dart
│   │   # Budget limits, progress bars, and over-budget warnings
│   └── reports_screen.dart
│       # Detailed analytics, category distribution, spending pie/bar charts
├── utils/
│   └── constants.dart
│       # Design tokens, color palette, typography styles, and currency symbol
├── widgets/
│   ├── budget_card.dart
│   │   # Reusable budget status card
│   ├── category_chart.dart
│   │   # Pie/Bar charts for financial distributions
│   ├── summary_card.dart
│   │   # Income/Expense balance summary widget
│   └── transaction_tile.dart
│       # Individual list item with merchant icon and category tags
├── main.dart
│   # App entry point, theme declaration, and floating dock navigation
├── test/
│   ├── income_test.dart
│   │   # SQLite income insertion and balance calculation test
│   ├── transaction_delete_revert_test.dart
│   │   # Unit tests verifying balance reversal upon deletion
│   └── widget_test.dart
│       # Widget UI tests, theme verification, and navigation smoke tests
├── pubspec.yaml
│   # Dependencies and asset declarations
```

```
|
└─ analysis_options.yaml
# Dart linter and static code quality rules
```

5. Key Features & Implementation Details

5.1 Neo-Banking Dark Theme

- **Color Scheme:** Deep pitch black background (`#0A0A0C`), card surfaces (`#141417`), high-contrast text (`#FFFFFF`, `#A1A1AA`), and vibrant status indicators.
- **Indian Rupee Currency (₹):** All figures are rendered with the Rupee symbol and two-decimal precision.
- **Floating Dock Bottom Bar:** Custom floating navigation dock hovering with pill styling and smooth tab switching.

5.2 Dashboard & Quick Operations

- **Hero Balance Card:** Prominently showcases net balance with account details and interactive dots menu.
- **Quick Action Triggers:** Four circular shortcuts:
 1. **+ Top up:** Opens Add Income modal.
 2. **↗ Move:** Opens Add Expense modal.
 3. **📄 Details:** Navigates to full transaction history.
 4. **⋮ More:** Displays quick account management sheet.
- **Recent Operations:** Grouped card showing latest transactions with custom merchant avatars and instant click-to-view details.

5.3 Add Income & Expense Workflow

- Segmented toggle to switch between **Income** and **Expense**.
- Dynamic category picker:
 - Salary
 - Investments
 - Freelance
 - Food
 - Shopping
 - Bills
 - Transport
 - Health
 - Entertainment
- Interactive date & time selector.

- Input validation ensuring amounts are positive numerical values.

5.4 Transaction Detail Modal & Deletion Reversion

- Tapping any transaction displays a rich receipt modal showing:
 - Merchant emblem, transaction type, timestamp, and signed amount (+₹... / -₹...).
 - Status badge (Completed).
 - Interactive toggles: "Exclude from analytics" and category switcher.
 - **Half-Year Spending Chart:** Dynamic 6-month comparison bar chart.
- **Reversion Engine:** Deleting a transaction immediately updates the database and recalculates the total balance and category spending accurately.

5.5 Budgets & Financial Planning

- Set monthly budget limits per category.
- Color-coded progress bars:
 - Green under 80%
 - Yellow 80–99%
 - Red for over-budget
- Quick budget creation dialog with pre-filled recommendations.

5.6 Reports & Analytical Insights

- Filter transactions by timeframe:
 - Weekly
 - Monthly
 - Yearly
- Category spending distribution pie chart.
- Income vs. Expense monthly progression bar graph.

6. Database Design (SQLite)

The local SQLite database (`finance_tracker.db`) consists of three primary tables.

6.1 transactions Table

```
CREATE TABLE transactions(
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  title TEXT NOT NULL,
  amount REAL NOT NULL,
  date TEXT NOT NULL,
  category TEXT NOT NULL,
```

```
type TEXT NOT NULL,      -- 'income' or 'expense'
merchant TEXT,
notes TEXT,
excludeFromAnalytics INTEGER DEFAULT 0
);
```

6.2 budgets Table

```
CREATE TABLE budgets(
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  category TEXT UNIQUE NOT NULL,
  amount REAL NOT NULL,
  month TEXT NOT NULL
);
```

6.3 goals Table

```
CREATE TABLE goals(
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  title TEXT NOT NULL,
  targetAmount REAL NOT NULL,
  savedAmount REAL NOT NULL,
  deadline TEXT
);
```

7. Testing & Quality Assurance

A comprehensive automated test suite covers unit logic, database transactions, widget rendering, and theme specifications.

7.1 Automated Test Results (10/10 Passed)

00:01 +1: Transaction Delete & Revert Back Unit Tests:
Deleting an Expense reverts expense total and net balance

00:01 +2: Transaction Delete & Revert Back Unit Tests:
Deleting an Income reverts income total and net balance

00:02 +3: TransactionsScreen Widget Tests:
Renders TransactionsScreen with summary card and transaction list

00:02 +4: TransactionsScreen Widget Tests:

TransactionTile renders formatted amount and triggers onDelete callback

00:02 +5: TransactionsScreen Widget Tests:

TransactionDetailScreen renders merchant, status, and half-year chart

00:02 +6: widget_test.dart:

Theme constants match neo-banking specifications

00:02 +7: widget_test.dart:

App launches with Slice UPI Dark Theme and 4 navigation tabs

00:03 +8: widget_test.dart:

SummaryCard renders high contrast text in dark mode

00:03 +9: widget_test.dart:

Remaining screens instantiate without errors

00:04 +10: income_test.dart:

Add income and verify in database

00:04 +10: All tests passed!

8. Setup & Execution Instructions

8.1 Prerequisites

- Flutter SDK (version 3.19.0 or later)
- Dart SDK (version 3.3.0 or later)
- Android Studio / Xcode / VS Code with Flutter extension
- Git CLI

8.2 Installation Steps

1. Clone the repository

git clone

<https://github.com/Vik-713/itm-flutter-miniproject-expense-tracker-budget-planning-application-10-09-2026.git>

2. Enter the Flutter project directory

cd

itm-flutter-miniproject-expense-tracker-budget-planning-application-10-09-2026/itm-flutter-finance-tracker

3. Fetch dependencies

flutter pub get

4. Run automated test suite

flutter test --concurrency=1

5. Launch the application

flutter run -d chrome # Run in web browser

OR

flutter run # Run on connected mobile device / simulator

9. Pull Request & Submission Information

- **Forked Repository:**
<https://github.com/Vik-713/itm-flutter-miniproject-expense-tracker-budget-planning-application-10-09-2026>
- **Upstream Repository:**
<https://github.com/engineersurajsahani/itm-flutter-miniproject-expense-tracker-budget-planning-application-10-09-2026>
- **Target Branch:** `main`
- **Submission Pull Request Link:**
Create Pull Request:
<https://github.com/engineersurajsahani/itm-flutter-miniproject-expense-tracker-budget-planning-application-10-09-2026/compare/main...Vik-713:itm-flutter-miniproject-expense-tracker-budget-planning-application-10-09-2026:main>

10. Conclusion

The **Expense Tracker & Budget Planning Application** fulfills all mini-project requirements and incorporates real-world fintech standards: offline data integrity via SQLite, a cohesive neo-banking user experience, verified automated test coverage, and clean Git version control with nested folder architecture.